

Assignment 3.1

K. Jyoshno Pranav

Batch:-46

2303A53016

1: Zero-Shot Prompting (Palindrome Number Program)

Prompt:-

1) Write a Python function `is_palindrome(num: int) -> bool` that determines if a given integer is a palindrome. A palindrome number reads the same forwards and backwards (e.g., 121, 3443).

Requirements:

- Input: Single integer num (can be positive, negative, or zero)
 - Output: True if palindrome, False otherwise
 - Handle edge cases: single digits, zero, negative numbers, numbers with leading zeros in string form
 - Do **not** convert to string—use mathematical operations only (reverse digits)
 - Time complexity: $O(\log n)$, Space complexity: $O(1)$
 - Include docstring and type hints. Code(Python):-
-

```
def is_palindrome(num: int) -> bool:    if num
< 0:
    return False    if
num < 10:        return
True    original_num =
num    reversed_num = 0
while num > 0:
    last_digit = num % 10    reversed_num =
reversed_num * 10 + last_digit    num //= 10    return
original_num == reversed_num if __name__
== "__main__":    test_cases = [
    (121, True),
```

```
(12321, True),
(1221, True),
(123, False),
(-121, False),
(0, True),
(5, True),
(10, False),
(101, True),
(1001, True),
(100, False),
(1234321, True),
(1234567, False),
(9999, True),
```

```
]
```

```
print("Testing is_palindrome function:")    print("-" * 40)
```

```
passed = 0    failed = 0    for test_num, expected in
```

```
test_cases:    result = is_palindrome(test_num)    status
```

```
= "✓ PASS" if result == expected else "X FAIL"    if result
```

```
== expected:
```

```
    passed += 1    else:
```

```
failed
```

```
+= 1
```

```
print(f"is_palindrome({test_num:8}) = {result:5} | Expected: {expected:5} | {status}")
```

```
print("-"
```

```
A-3.1.py X Hydra Tool USAGE.txt
A-3.1.py > ...
1 def is_palindrome(num: int) -> bool:
2     if num < 0:
3         return False
4     if num < 10:
5         return True
6     original_num = num
7     reversed_num = 0
8     while num > 0:
9         last_digit = num % 10
10        reversed_num = reversed_num * 10 + last_digit
11        num //= 10
12    return original_num == reversed_num
13 if __name__ == "__main__":
14     test_cases = [
15         (121, True),
16         (12321, True),
17         (1221, True),
18         (123, False),
19         (-121, False),
20         (0, True),
21         (5, True),
22         (10, False),
23         (101, True),
24         (1001, True),
25         (100, False),
26         (1234321, True),
27         (1234567, False),
28         (9999, True),
29     ]
30     print("Testing is_palindrome function:")
31     print("-" * 40)
32     passed = 0
```

```
* 40)
```

```
print(f"Results:
```

```
{passed}
```

```
passed,
```

```
{failed} failed")
```

Sample output:-

Testing is_palindrome function:

is_palindrome(121) = True | Expected: True | ✓ PASS is_palindrome(

12321) = True | Expected: True | ✓ PASS is_palindrome(1221) =

True | Expected: True | ✓ PASS is_palindrome(123) = False |

Expected: False | ✓ PASS is_palindrome(-121) = False | Expected: False

| ✓ PASS is_palindrome(0) = True | Expected: True | ✓ PASS

is_palindrome(

5) = True | Expected: True | ✓ PASS is_palindrome(10) = False |

Expected: False | ✓ PASS is_palindrome(101) = True | Expected:

True | ✓ PASS is_palindrome(1001) = True | Expected: True | ✓ PASS

is_palindrome(100) = False | Expected: False | ✓ PASS is_palindrome(

1234321) = True | Expected: True | ✓ PASS is_palindrome(1234567) =

False | Expected: False | ✓ PASS is_palindrome(9999) = True |

Expected: True | ✓ PASS

Results: 14 passed, 0 failed

2: One-Shot Prompting (Factorial Calculation)

Prompt:-

Write a Python function factorial(n) that calculates the factorial of a non-negative integer n. The factorial of n (denoted as n!) is the product of all positive integers less than or equal to n.

Provide one example: For input 5, the output should be 120 because $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Requirements:

- Handle $n = 0$ ($0! = 1$ by definition)
 - Handle negative inputs (return appropriate error)
 - Use iterative approach (not recursive)
 - Include proper type hints and docstring
- Code(Python):-
- ```
def factorial_one_shot(n: int) -> int:
 if n < 0:
 raise ValueError("Factorial is not defined for negative numbers")
```

```
 if n == 0 or n == 1:
 return 1
 result = 1
 for i in range(2, n + 1):
```

```

1): result *= i return

result def

factorial_zero_shot(n):

if n < 0: return

"Undefined" elif n == 0: return

1 else:

 result = 1

 for i in range(1, n + 1):

result *= i return result

print("=" * 60)

print("FACTORIAL CALCULATION - ONE-SHOT VS ZERO-SHOT COMPARISON")

print("=" * 60)

print("\nONE-SHOT SOLUTION (with example 5 → 120):")

print("-" * 40) test_cases = [5, 0, 1, 7, -3, 4] for n in test_cases:

 try:

 result = factorial_one_shot(n)

print(f"factorial_one_shot({n}) = {result}") except ValueError as

e:

 print(f"factorial_one_shot({n}) raises: {e}")

print("\nZERO-SHOT SOLUTION (no example provided):")

print("-" * 40) for n in test_cases: result =

factorial_zero_shot(n) print(f"factorial_zero_shot({n}) =

{result}")

print("\n" + "=" * 60)

print("COMPARISON ANALYSIS")

print("=" * 60)

```

```
print("\n1 CODE QUALITY:") print("One-shot: Type hints, Proper error handling, Efficient loop
(starts from 2)") print("Zero-shot: No type hints, Returns string for errors, Less efficient loop
(starts from 1)")
```

```
print("\n2 EDGE CASE HANDLING:") print("One-shot: n=0 returns 1, n=1 returns 1,
Negative raises ValueError") print("Zero-shot: n=0 returns 1, n=1 inefficient, Negative returns
string (mixed types)")
```

```
print("\n3 DOCUMENTATION:") print("One-shot: Complete docstring with examples, parameters,
and exceptions") print("Zero-shot: Minimal or no documentation")
```

```
print("\n4 TEST OUTPUTS:") print("One-shot: Returns int or raises exception
(consistent behavior)") print("Zero-shot: Returns int OR string
(inconsistent return types)") print("\n" + "=" * 60)
```

```
print("PERFORMANCE COMPARISON")
```

```
print("=" * 60)
```

```
import time def time_function(func, n, iterations=100000):
 start = time.perf_counter() for _ in range(iterations): if n >=
0: func(n) end = time.perf_counter() return end - start n
= 10 iterations = 100000 one_shot_time =
time_function(factorial_one_shot, n, iterations) zero_shot_time =
time_function(factorial_zero_shot, n, iterations)
print(f"\nPerformance test for n={n} (averaged over {iterations}
iterations):") print(f"One-shot solution time: {one_shot_time:.6f}
seconds") print(f"Zero-shot solution time: {zero_shot_time:.6f}
seconds") if one_shot_time < zero_shot_time:
 improvement = ((zero_shot_time - one_shot_time) / zero_shot_time) * 100 print(f"One-shot is
{improvement:.2f}% faster") else:
 print(f"Zero-shot is {((one_shot_time - zero_shot_time)/one_shot_time)*100:.2f}% faster")
```

```
print("\n" + "=" * 60)
```

```
print("CONCLUSION") print("=" * 60) print("One-shot prompting produces more
robust, efficient, and professional code") print("The example (5 → 120) helped the AI
understand edge cases and requirements") print("One-shot solution follows Python
best practices and has better error handling") Output:-
```

## FACTORIAL CALCULATION - ONE-SHOT VS ZERO-SHOT COMPARISON

◇ ONE-SHOT SOLUTION (with example 5 → 120):

-----

```
factorial_one_shot(5) = 120 factorial_one_shot(0) = 1 factorial_one_shot(1) =
1 factorial_one_shot(7) = 5040 factorial_one_shot(-3) raises: Factorial is not
defined for negative numbers
```

◇ ZERO-SHOT SOLUTION (no example provided):

-----

```
factorial_zero_shot(5) = 120 factorial_zero_shot(0)
```

```
A-3.1.py X Hydra Tool USAGE.txt
A-3.1.py > ...
1 def factorial_one_shot(n: int) -> int:
2 if n < 0:
3 raise ValueError("Factorial is not defined for negative numbers")
4
5 if n == 0 or n == 1:
6 return 1
7
8 result = 1
9 for i in range(2, n + 1):
10 result *= i
11
12 return result
13
14 def factorial_zero_shot(n):
15 if n < 0:
16 return "Undefined"
17 elif n == 0:
18 return 1
19 else:
20 result = 1
21 for i in range(1, n + 1):
22 result *= i
23 return result
24
25 print("=" * 60)
26 print("FACTORIAL CALCULATION - ONE-SHOT VS ZERO-SHOT COMPARISON")
27 print("=" * 60)
28
29 print("\nONE-SHOT SOLUTION (with example 5 → 120):")
30 print("-" * 40)
31 test_cases = [5, 0, 1, 7, -3, 4]
```

= 1 factorial\_zero\_shot(1)



= 1 factorial\_zero\_shot(7) = 5040 factorial\_zero\_shot(-3)

= Undefined

---

### 3: Few-Shot Prompting (Armstrong Number Check)

Prompt:-

Write a Python function that checks whether a given number is an Armstrong number. An Armstrong number is a number that is equal to the sum of its own digits each raised to the power of the number of digits.

Code(Python):- def is\_armstrong(number):

# Input validation if not isinstance(number,

int):

return "Invalid input: not an integer" if number

< 0:

return "Invalid input: negative number"

digits = str(number) power = len(digits) total

= sum(int(digit) \*\* power for digit in digits) if total

== number:

return "Armstrong Number"

else:

return "Not an Armstrong Number" print(is\_armstrong(153)) #

Armstrong Number print(is\_armstrong(370)) # Armstrong

Number print(is\_armstrong(123)) # Not an Armstrong Number

competive progrmaing.py \* × <untitled> \* ×

```
1 def is_armstrong(number):
2 # Input validation
3 if not isinstance(number, int):
4 return "Invalid input: not an integer"
5 if number < 0:
6 return "Invalid input: negative number"
7
8 digits = str(number)
9 power = len(digits)
10 total = sum(int(digit) ** power for digit in digits)
11
12 if total == number:
13 return "Armstrong Number"
14 else:
15 return "Not an Armstrong Number"
16 print(is_armstrong(153)) # Armstrong Number
17 print(is_armstrong(370)) # Armstrong Number
18 print(is_armstrong(123)) # Not an Armstrong Number
19 |
```

Shell ×

>>> %Run -c \$EDITOR\_CONTENT

```
Armstrong Number
Armstrong Number
Not an Armstrong Number
```